Парсер HTML в JSON 7 Share

делаю тестовое, парсер html в json на чистом js, DOMParser нельзя
функция html2json(htmlText) лежит в html2json.js, файл подключается в index.html обычным скриптом, не модулем, так что export туда нельзя
в package.json стоит type: module
хочу сделать test_samples.js который берет все .html из папки html_samples и прогоняет через функцию, чтобы видеть что ничего не падает. как его подключить к html2json.js если там нет экспорта?

Написание скрипта для прогона примеров через html2json

Самый чистый вариант без правки `html2json.js`: прочитать файл как текст и выполнить его через встроенный модуль `node:vm` как обычный скрипт. В конец кода дописывается `;html2json;`, и значение этого последнего выражения возвращается из `runInThisContext`. Так ты получаешь функцию, и неважно, объявлена она через `function` или `const`.

Created a file, shared a file

Ключевая часть в `test_samples.js` вот эта:

```js
const code = fs.readFileSync(libPath, 'utf8');
const html2json = vm.runInThisContext(`${code}\n;html2json;`, { filename: libPa
```

Запуск: `node test_samples.js`, или добавь в package.json `"test:samples": "node test_samples.js"`. Скрипт проходит по всем `.html` в `html_samples`, ловит ошибки и выводит стек для упавших файлов. Ещё он проверяет, что результат проходит через `JSON.stringify`, чтобы поймать циклические ссылки. Если что-то упало, код выхода будет 1.

Пара нюансов:

- `filename` в опциях нужен, чтобы в стеке ошибок были нормальные номера строк из `html2json.js`. Строки не сдвигаются, потому что `;html2json;` дописывается в самый конец.
- Если внутри `html2json.js` на верхнем уровне есть обращения к `document` или `window`, в Node это упадёт. Такой код придётся обернуть в `if (typeof document !== 'undefined')`.

Если менять `html2json.js` всё-таки можно, есть альтернатива: добавить в конец файла `globalThis.html2json = html2json;`, а в тесте сделать `import './html2json.js'`. Браузеру это не мешает. Но тогда Node загрузит файл как ES-модуль в strict mode, и нестрогий код (например, присваивание необъявленной переменной) начнёт падать. Вариант с `vm` такой проблемы не имеет.

Test samples
JS

теперь нужны html примеры для html_samples, по одному файлу на каждый случай, чтобы если упадет было понятно на чем
назови файлы с номерами типа 01_simple.html, 02_..., и в начале каждого напиши комментарий что он проверяет

проанализируй все возможные случаи

и для каждого файла коротко напиши как браузер должен это понять

Разбирая сверку сгенерированных страниц со спецификацией парсера

Соберу набор: по файлу на каждый случай, с комментарием в начале (что проверяет и как это понимает браузер), плюс README с общей таблицей. Поведение браузера сверю с parse5, это парсер, который точно следует спецификации HTML.

Created a file, shared a file, ran 5 commands

Готово: в `html_samples.zip` лежат 79 файлов от `01_simple.html` до `79_real_page.html` и `README.md` с таблицей "файл / что проверяет / как понимает браузер". В начале каждого файла есть комментарий с двумя строками, ТЕСТ и БРАУЗЕР. Все описания я сверил с parse5, который разбирает HTML строго по спецификации, как Chrome и Firefox.

Группы файлов:

- 01-12, **документ:** пустой файл, только комментарий, только текст, неявные html/head/body, разные doctype, quirks mode, BOM, CRLF, NUL.
- 13-21, **теги:** регистр, void-элементы, `/>` у обычных элементов, `</br>` и `</p>` без пары, неизвестные теги, `<` в тексте, оборванный тег в конце файла.
- 22-29, **атрибуты:** кавычки, пустые значения, дубли, пробелы, странные имена вроде `@click` и `:class`, сущности в значениях, сломанные атрибуты, незакрытая кавычка.
- 30-35, **текст и сущности:** именованные, числовые и битые сущности, пробелы, юникод, перевод строки после `pre` и `textarea`.
- 36-40, **комментарии:** обычные, оборванные, bogus (`<?xml`, CDATA), условные IE, незакрытый.
- 41-49, **сырой текст:** script, style, title, textarea, xmp, iframe, plaintext, `</script>` внутри строки.
- 50-63, **автозакрытие и таблицы:** p, li, dd, option, вложенные a/form/button, adoption agency, tbody, foster parenting.
- 64-76, **особые случаи:** head и body не на своих местах, svg, MathML, template, select, ruby, frameset.

- **77-79, нагрузка и интеграция:** 3000 уровней вложенности, файл на 870 КБ, реалистичная страница.

Что чаще всего роняет самописный парсер:

- `31_entities_numeric`: `String.fromCodePoint(0x110000)` бросает RangeError, если не проверять диапазон.
- `77_deep_nesting`: рекурсивный парсер может упасть с переполнением стека.
- `02_empty`, `21`, `29`, `40`, `48`: пустой ввод и конец файла посреди конструкции. Это частые места для бесконечного цикла или обращения к `undefined`.
- `09_bom`: `readFileSync` с `'utf8'` оставляет BOM первым символом строки.

Учти, что колонка "как понимает браузер" описывает полный алгоритм из спецификации. Для тестового задания вряд ли нужно повторять adoption agency (59, 60) или foster parenting (62) один в один, главное, чтобы на этих файлах ничего не падало. В `02_empty.html` комментария нет, иначе файл перестал бы быть пустым. `46` (noscript) и `73` (select) у разных браузеров разбираются по-разному, это отмечено в их комментариях.

Html samples
ZIP

вот моя функция, сейчас для <p class="x"> имя тега получается 'p class="x"'
допиши, чтобы имя и атрибуты были отдельно, атрибуты через регулярку в отдельной функции parseAttributes должны работать class="x", id='y', n=5 и disabled без значения
и регулярку объясни по частям

```
function html2json(htmlText) {
    const tokens = [];
    let i = 0;

    while (i < htmlText.length) {
        if (htmlText.startsWith('</', i)) {
            const end = htmlText.indexOf('>',
i);
            const name = htmlText
                .slice(i + 2, end)
                .trim()
                .toLowerCase();
            tokens.push({ type: 'endTag', name
});
            i = end + 1;
        } else if (htmlText[i] === '<') {
            const end = htmlText.indexOf('>',
i);
            const inside = htmlText.slice(i +
1, end);
            const name = inside.split(' ')
[0].toLowerCase();

            tokens.push({ type: 'startTag',
name });
            i = end + 1;
        } else {
            let end = htmlText.indexOf('<',
i);
            if (end === -1) end =
htmlText.length;
            tokens.push({ type: 'text', value:
htmlText.slice(i, end) });
            i = end;
        }
    }

    return tokens;
}
```

Show more

Test the attribute regex on sample tags ▸

Проверил на твоих случаях и на нескольких файлах из `html_samples`, всё работает. Изменился только блок открывающего тега, плюс добавилась функция `parseAttributes`:

javascript

```
const ATTR_RE = /([^\s"'>\/=]+)(?:\s*=\s*(?:"([^"]*)"|'([^']*)'|([^\s>]+)))?/g;

function parseAttributes(str) {
    const attributes = {};

    for (const match of str.matchAll(ATTR_RE)) {
        const name = match[1].toLowerCase();
        // при повторе атрибута браузер оставляет первый
        if (Object.hasOwn(attributes, name)) continue;
        // сработает только одна группа значения, если ни одна - атрибут без зн
        attributes[name] = match[2] ?? match[3] ?? match[4] ?? '';
    }

    return attributes;
}

function html2json(htmlText) {
    const tokens = [];
    let i = 0;

    while (i < htmlText.length) {
        if (htmlText.startsWith('</', i)) {
            const end = htmlText.indexOf('>', i);
            const name = htmlText
                .slice(i + 2, end)
                .trim()
                .toLowerCase();
            tokens.push({ type: 'endTag', name });
            i = end + 1;
        } else if (htmlText[i] === '<') {
            const end = htmlText.indexOf('>', i);
            const inside = htmlText.slice(i + 1, end);

            // имя тега: всё до первого пробельного символа, "/" или ">"
            const nameMatch = inside.match(/^[^\s\/>]+/);
            const name = nameMatch ? nameMatch[0].toLowerCase() : '';
            // всё, что после имени, это атрибуты
            const attributes = parseAttributes(inside.slice(nameMatch ? nameMat

            tokens.push({ type: 'startTag', name, attributes });
            i = end + 1;
        } else {
            let end = htmlText.indexOf('<', i);
            if (end === -1) end = htmlText.length;
            tokens.push({ type: 'text', value: htmlText.slice(i, end) });
```

```
            i = end;
        }
    }

    return tokens;
}
```

Результат:

```
p class="x"                  -> { name: 'p', attributes: { class: 'x' } }
div id='y' n=5 disabled      -> { name: 'div', attributes: { id: 'y', n: '5',
disabled: '' } }
img src="a.png" /            -> { name: 'img', attributes: { src: 'a.png' } }
```

Почему ломалось раньше: `split(' ')` делит только по обычному пробелу. Если после имени стоит перевод строки или таб (`<p\n  class="x">`), строка не делится, и в имя попадает всё целиком. `[^\s\/>]+` останавливается на любом пробельном символе.

Регулярка по частям

```
([^\s"'>\/=]+)  (?:  \s*=\s*  (?:  "([^"]*)"  |  '([^']*)'  |  ([^\s>]+)  )
)?
 └── 1: имя ──┘       └ = ┘        └ 2 ──┘     └ 3 ──┘     └── 4 ──┘
               └──────────── необязательная часть "=значение" ────────────┘
```

`([^\s"'>\/=]+)`, **группа 1, имя атрибута.** `[^...]` означает "любой символ, кроме перечисленных", а `+` означает "один или больше". Исключены пробельные символы `\s`, кавычки, `>`, `/` и `=`, то есть всё, что может стоять между атрибутами или после имени. Поэтому `class`, `data-id`, `@click`, `:class` подходят, а одиночный `/` в `<br />` никогда не станет атрибутом.

`(?: ... )?`, **необязательный хвост со значением.** `(?: )` это группа без сохранения: она только объединяет кусок, чтобы повесить на него `?` (“0 или 1 раз”). Если хвоста нет, получается атрибут без значения, как `disabled`.

`\s*=\s*`, **знак равенства.** Пробелы вокруг `=` разрешены (`id = "x"`), поэтому `\s*`, то есть "ноль или больше пробельных символов".

Внутренний `(?: A | B | C )`, **три формы значения.** `|` означает "или". Пробуются по порядку, срабатывает первая подходящая:

- `"([^"]*)"`, **группа 2, значение в двойных кавычках.** Открывающая кавычка, потом любые символы, кроме `"`, потом закрывающая. Внутри можно всё, включая пробелы, `>` и одинарные кавычки: `title="it's"`. `*` вместо `+`, чтобы работало и пустое `value=""`.
- `'([^']*)'`, **группа 3, значение в одинарных кавычках.** То же самое зеркально: `id='y'`, внутри можно `"`.

- `([^\s>]+)`, **группа 4, значение без кавычек.** Всё до первого пробела или `>`: `n=5`, `href=/a/b`. Так же себя ведёт и браузер.

Кавычки стоят вне скобок групп 2 и 3, поэтому в результат попадает только содержимое, без кавычек.

`g`, **флаг global.** Без него регулярка найдёт только первый атрибут. С ним `matchAll` возвращает все совпадения по очереди. Пробелы между атрибутами специально не описаны: регулярка их просто пропускает, пока не найдёт следующее имя. Поэтому работает и `a="1"b="2"` без пробела.

`match[2] ?? match[3] ?? match[4] ?? ''`. У каждого совпадения сработает максимум одна из групп 2-4, остальные будут `undefined`. `??` берёт первую не `undefined`, а если значения не было вовсе, ставит `''`, как это делает браузер для `disabled`.

Две детали в `parseAttributes`. `Object.hasOwn` вместо `name in attributes` нужен потому, что `in` смотрит и в прототип. Для атрибута с именем `constructor` или `toString` проверка сказала бы "уже есть", и атрибут потерялся бы. Регулярку можно держать в константе с флагом `g` снаружи функции, потому что `matchAll` работает с копией и не ломается от сохранённого `lastIndex`.

Сущности вроде `&` в значениях пока не декодируются, их обычно разбирают отдельной функцией (файл `27_attr_entities.html` как раз про это).

на 21 семпле, скрипт завис, потому что не мог найти закрывающую скобку, проверь и скажи что исправить

```
const ATTR_RE = /([^\s"'>\/=]+)(?:\s*=\s*(?:"
([^"]*)"|'([^']*)'|([^\s>]+)))?/g;

function parseAttributes(str) {
    const attributes = {};

    for (const match of str.matchAll(ATTR_RE))
{
        const name = match[1].toLowerCase();
        // при повторе атрибута браузер
оставляет первый
        if (Object.hasOwn(attributes, name))
continue;
        // сработает только одна группа
значения, если ни одна - атрибут без значения
        attributes[name] = match[2] ??
match[3] ?? match[4] ?? '';
    }

    return attributes;
}

function html2json(htmlText) {
    const tokens = [];
    let i = 0;

    while (i < htmlText.length) {
        if (htmlText.startsWith('</', i)) {
            const end = htmlText.indexOf('>',
i);
            const name = htmlText
                .slice(i + 2, end)
                .trim()
                .toLowerCase();
            tokens.push({ type: 'endTag', name
});
            i = end + 1;
        } else if (htmlText[i] === '<') {
            const end = htmlText.indexOf('>',
i);
            const inside = htmlText.slice(i +
1, end);

            const nameMatch =
inside.match(/^[^\s\/>]+/);
```

```
            const name = nameMatch ?
nameMatch[0].toLowerCase() : '';
            // всё, что после имени, это
атрибуты
            const attributes =
parseAttributes(
                inside.slice(nameMatch ?
nameMatch[0].length : 0)
            );
            tokens.push({ type: 'startTag',
name, attributes });
            i = end + 1;
        } else {
            let end = htmlText.indexOf('<',
i);
            if (end === -1) end =
htmlText.length;
            tokens.push({ type: 'text', value:
htmlText.slice(i, end) });
            i = end;
        }
    }

    return tokens;
}
```

Show more

Ran 2 commands ▸

Зависание даёт строка `i = end + 1`, когда `>` не найден.

В конце `21_unterminated_tag_eof.html` стоит `<div class="never-closed` без `>`. На нём происходит вот что:

1. `htmlText.indexOf('>', i)` не находит скобку и возвращает `-1`.
2. `i = end + 1` превращается в `i = -1 + 1`, то есть `i = 0`.
3. Цикл начинает разбор файла с самого начала, снова доходит до того же места и снова прыгает в 0. Так бесконечно, а `tokens` всё растёт, пока не кончится память.

Та же проблема есть и в ветке закрывающего тега: на `</div` в конце файла будет то же самое.

Исправление: после каждого `indexOf('>')` проверять `-1` и выходить из цикла. Браузер ведёт себя так же: тег, оборванный концом файла, отбрасывается целиком.

javascript

```
        if (htmlText.startsWith('</', i)) {
            const end = htmlText.indexOf('>', i);
            // тег оборван концом файла - браузер его отбрасывает
            if (end === -1) break;
            const name = htmlText
                .slice(i + 2, end)
                .trim()
                .toLowerCase();
            tokens.push({ type: 'endTag', name });
            i = end + 1;
        } else if (htmlText[i] === '<') {
            const end = htmlText.indexOf('>', i);
            // тег оборван концом файла - браузер его отбрасывает
            if (end === -1) break;
            const inside = htmlText.slice(i + 1, end);
            // ... дальше без изменений
```

`break` здесь безопасен: если `>` нет до конца файла, после этого места разбирать больше нечего.

С этой правкой я прогнал твой код по всем 79 файлам. Ничего не падает и не зависает, а для 21-го последние токены как у браузера: `p`, `"before"`, `/p`, `"\n"`, без обрывка div. Других файлов, где `<` остаётся без `>`, в наборе нет, так что зависал только этот.

Следующее, что всплывёт на той же строке: `indexOf('>')` не учитывает кавычки. В `22_attr_quotes.html` есть `e="has > inside"`, и тег обрежется на `>` внутри значения. В `29_attr_unterminated_quote_eof.html` наоборот: браузер считает, что незакрытая кавычка тянется до конца файла, а у тебя тег закроется на первом `>`. Чинится отдельной функцией поиска конца тега, которая пропускает `>` внутри кавычек. Это не зависание, а неверный результат, так что скрипт сейчас эти файлы пройдёт.

у меня есть константа TEXT_TAGS = ['script', 'style', 'textarea']
нужно в splitIntoTokens после открывающего тега из этого списка взять все до закрывающего тега как текст, даже если внутри есть "<" или " "
закрывающий тег может быть в любом регистре: </script>, </SCRIPT>
если закрывающего тега нет - текст до конца строки

вот моя функция:

```
function splitIntoTokens(text) {
    const tokens = [];
    let i = 0;

    while (i < text.length) {
        if (text.startsWith('<!--', i)) {
            // если омментарий до "-->",
            // а если его нет - до конца
текста

            let end = text.indexOf('-->', i +
4);
            if (end === -1) end = text.length;
            tokens.push({ type: 'comment',
value: text.slice(i + 4, end) });
            i = end + 3;
        } else if (text.startsWith('<!', i)) {
            // <!DOCTYPE html>

            let end = text.indexOf('>', i);
            if (end === -1) end = text.length;
            const content = text.slice(i + 2,
end).trim();
            if
(content.toLowerCase().startsWith('doctype')) {
                tokens.push({ type: 'doctype',
value: content.slice(7).trim() });
            }
            i = end + 1;
        } else if (text.startsWith('</', i)) {
            const end = text.indexOf('>', i);
            // тег оборван концом файла -
браузер его отбрасывает
            if (end === -1) break;
            const name = text
                .slice(i + 2, end)
                .trim()
                .toLowerCase();
            tokens.push({ type: 'endTag', name
});
            i = end + 1;
        } else if (text[i] === '<' && /[a-zA-
Z]/.test(text[i + 1] || '')) {
            const end = text.indexOf('>', i);
            // тег оборван концом файла -
```

```
браузер его отбрасывает
            if (end === -1) break;
            const inside = text.slice(i + 1,
end);

            const nameMatch =
inside.match(/^[^\s\/>]+/);
            const name = nameMatch ?
nameMatch[0].toLowerCase() : '';

            // всё, что после имени, это
атрибуты
            const attributes = parseAttributes(
                inside.slice(nameMatch ?
nameMatch[0].length : 0)
            );
            tokens.push({ type: 'startTag',
name, attributes });
            i = end + 1;

            if (TEXT_TAGS.includes(name)) {
                // ищем закрывающий тег с text
тегов
            }
        } else {
            let end = text.indexOf('<', i + 1);
            if (end === -1) end = text.length;
            tokens.push({ type: 'text', value:
text.slice(i, end) });
            i = end;
        }
    }

    return tokens;
}
```

Show more

Ran a command ▸

Замени пустой блок `if (TEXT_TAGS.includes(name))` на этот:

javascript

```
            if (TEXT_TAGS.includes(name)) {
                // ищем </name, за которым идет пробел, "/" или ">", в любом ре
                const closeRe = new RegExp(`</${name}[\\s/>]`, 'gi');
                closeRe.lastIndex = i; // искать с позиции сразу после открываю
                const match = closeRe.exec(text);

                // нет закрывающего тега - текст до конца строки
                const textEnd = match ? match.index : text.length;
                if (textEnd > i) {
                    tokens.push({ type: 'text', value: text.slice(i, textEnd) }
                }
                // сам закрывающий тег не трогаем: на следующем шаге цикла
                // его разберет обычная ветка '</'
                i = textEnd;
            }
```

Остальную функцию менять не нужно. Я прогнал её по всем 79 файлам: ничего не падает, весь набор разбирается примерно за 0.1 секунды.

Как это работает:

- `</${name}[\\s/>]` ищет `</script`, после которого стоит пробельный символ, `/` или `>`. Без этой последней части `</scriptx>` или `</scripts>` внутри кода закрыли бы тег. В JS-строке слеш перед `s` удваивается, поэтому `\\s`.
- **Флаг** `i` отвечает за регистр, так что подходят `</script>`, `</SCRIPT>`, `</Script >`.
- **Флаг** `g` нужен не для поиска всех совпадений, а для `lastIndex`: только с `g` (или `y`) метод `exec` начинает поиск с заданной позиции. Иначе он искал бы с начала строки и нашёл бы чужой закрывающий тег выше по тексту.
- `name` **в регулярку подставлять безопасно**: он может быть только одним из трёх слов из `TEXT_TAGS`, спецсимволов в нём нет.
- `if (textEnd > i)` нужен, чтобы `<script></script>` не создавал пустой текстовый токен.

Что получилось на файлах из набора:

- **41, 44:** `<`, `&&` и `</div>` внутри script и style остаются текстом.
- **42:** `</SCRIPT>` и `</script >` закрывают тег. `"</script>"` внутри JS-строки тоже закрывает его раньше времени, и браузер делает так же.
- **47:** `</scriptx>` и `</scrip>` script не закрывают.
- **48:** без закрывающего тега весь хвост файла уходит в текст script.

Ещё одна проблема видна на 42-м файле, правда, в ветке закрывающего тега, а не в новом коде. Для `</script foo="bar">` получается `name: 'script foo="bar"'`, то есть ровно та же ошибка, что была у открывающих тегов. Чинится так же:

```javascript
const name = (text.slice(i + 2, end).match(/^[^\s\/>]*/)[0]).toLowe
```

Здесь `*` вместо `+`: для `</>` получится пустое имя, и `[0]` не упадёт на `null`.

По спецификации у textarea есть две особенности, которых сейчас нет. Сущности внутри неё декодируются (`<` превращается в `<`, в отличие от script и style), и первый перевод строки сразу после `<textarea>` отбрасывается. Если в задании этого не требуют, можно не делать: на "ничего не падает" это не влияет.

Pasted text.txt

TXT

вот мой парсер сейчас. на каком валидном html он упадет, зависнет или выдаст неправильное дерево? дай пару примеров

Ran 6 commands

Я прогнал твой парсер по всем 79 файлам и по отдельным валидным примерам. Зависать ему негде: каждая ветка цикла сдвигает `i` вперёд. Упал он один раз, и не сам парсер, а `JSON.stringify` (пункт 6). Основная проблема в другом: на вполне валидном HTML получается неправильное дерево.

1. Необязательные закрывающие теги

По спецификации можно не писать `</li>`, `</p>`, `</td>`, `</tr>`, `</dt>`, `</dd>`, `</option>`. Это валидный HTML, и встречается он часто.

```
<ul><li>One<li>Two<li>Three</ul>
у тебя:   ul[li["One", li["Two", li["Three"]]]]      каждый li вложен в
предыдущий
браузер:  ul[li["One"], li["Two"], li["Three"]]

<p>Intro<div>Block</div>
у тебя:   p["Intro", div["Block"]]
браузер:  p["Intro"], div["Block"]                   блочный элемент
закрывает p
```

Исправление: перед тем как открыть новый тег, закрыть то, что он неявно закрывает. Важно, что `parent` теперь нужно вычислять после этого шага:

javascript

```
const AUTO_CLOSE = {
    li: ['li'],
    dt: ['dt', 'dd'],
    dd: ['dt', 'dd'],
    option: ['option'],
    tr: ['tr', 'td', 'th'],
    td: ['td', 'th'],
    th: ['td', 'th'],
    p: ['p']
};
const P_CLOSERS = ['div', 'ul', 'ol', 'dl', 'table', 'h1', 'h2', 'h3', 'h4', 'h
    'section', 'article', 'header', 'footer', 'nav', 'main', 'aside', 'form', '
    'blockquote', 'hr', 'figure'];

// в buildTree
for (const token of tokens) {
    if (token.type === 'startTag') {
        // while, а не if: <tr> после <td> должен закрыть и td, и предыдущий tr
        while (AUTO_CLOSE[token.name]?.includes(stack[stack.length - 1].tag)) s
        if (stack[stack.length - 1].tag === 'p' && P_CLOSERS.includes(token.nam
    }
    const parent = stack[stack.length - 1];
    // ... дальше как было
```

Браузер проверяет это немного сложнее, чем по одному верхнему элементу стека, но для нормальной разметки такой проверки хватает.

2. `>` внутри значения атрибута

```
<a href="/x" title="a > b">link</a>
у тебя:   a{ href: "/x", title: "\"a" }[" b\">link"]
браузер:  a{ href: "/x", title: "a > b" }["link"]
```

Причина в том, что `indexOf('>')` не знает про кавычки. Для открывающего тега его нужно заменить регуляркой, которая перешагивает строки в кавычках:

javascript

```
const TAG_END_RE = /(?:"[^"]*"|'[^']*'|[^"'>])*>/y;

// вместо const end = text.indexOf('>', i);
TAG_END_RE.lastIndex = i;
const end = TAG_END_RE.exec(text) ? TAG_END_RE.lastIndex - 1 : -1;
```

Разбор регулярки: строка в двойных кавычках, или строка в одинарных, или любой символ, кроме кавычки и `>`, повторить сколько угодно раз, потом `>`. Флаг `y` (sticky) требует, чтобы совпадение начиналось ровно с `lastIndex`, а не где-то дальше в тексте. Если кавычка не закрыта, совпадения нет, и `end === -1`. Браузер в этом случае тоже выбрасывает тег до конца файла.

3. Пропадают пробелы между словами

`if (token.value.trim() === '') continue;` выкидывает любой пробельный текст, а не только переносы строк из форматирования:

```
<p><b>Hello</b> <i>world</i></p>
у тебя:   p[b["Hello"], i["world"]]          при выводе получится
"Helloworld"
браузер:  p[b["Hello"], " ", i["world"]]

<pre>  ...  </pre> из одних пробелов и переносов: у тебя содержимое просто
исчезает
```

Компромисс: выбрасывать только пробельные узлы, где есть перенос строки (это отступы в коде), и никогда не трогать их внутри `pre`:

javascript

```
if (token.value.trim() === '' && token.value.includes('\n') && !stack.some(el =
```

Это осознанное решение по структуре JSON, и в задании прямо просят такие решения объяснять (Comment 2). Стоит написать в README, что пробельные узлы с переносом строк выбрасываются как форматирование.

4. Самозакрывающиеся теги в SVG

В HTML `<div/>` не закрывает тег, а внутри `<svg>` и `<math>` запись `/>` его закрывает. Inline SVG иконки встречаются на каждом втором сайте:

```
<svg><circle r="5"/><rect width="5"/></svg>
у тебя:   svg[circle[rect]]          rect оказался внутри circle
браузер:  svg[circle, rect]
```

javascript

```
// в splitIntoTokens, рядом с attributes
const selfClosing = /\/\s*$/.test(inside);
tokens.push({ type: 'startTag', name, attributes, selfClosing });

// в buildTree вместо if (!VOID_TAGS.includes(token.name)) stack.push(element);
const inForeign = token.name === 'svg' || token.name === 'math' ||
    stack.some(el => el.tag === 'svg' || el.tag === 'math');
const closed = VOID_TAGS.includes(token.name) || (inForeign && token.selfClosin
if (!closed) stack.push(element);
```

5. Сущности не декодируются

Это видно прямо на примере из твоего задания (`showExample1`): `<p>© 2024 My Website</p>` превращается в текст `"© 2024 My Website"`, а браузер покажет `"© 2024 My Website"`. Минимальная функция:

javascript

```
const ENTITIES = { amp: '&', lt: '<', gt: '>', quot: '"', apos: "'", nbsp: '\u0

function decodeEntities(str) {
    return str.replace(/&(#[xX][0-9a-fA-F]+|#\d+|[a-zA-Z]+);/g, (match, entity)
        if (entity[0] === '#') {
            const isHex = entity[1] === 'x' || entity[1] === 'X';
            const code = parseInt(entity.slice(isHex ? 2 : 1), isHex ? 16 : 10)
            // fromCodePoint бросает RangeError на числах больше 0x10FFFF
            const valid = code > 0 && code <= 0x10ffff && !(code >= 0xd800 && c
            return valid ? String.fromCodePoint(code) : '\uFFFD';
        }
        return ENTITIES[entity] ?? match; // неизвестную сущность оставляем как
    });
}
```

Её нужно применять к обычному тексту, к `textarea` и к значениям атрибутов, но не к `script` и `style`. Сущностей всего больше двух тысяч, поэтому полную таблицу обычно не тащат, а в README пишут, какие поддерживаются.

6. Очень глубокая вложенность

На 3000 вложенных `div` парсер отрабатывает нормально, но `JSON.stringify` падает с `Maximum call stack size exceeded`, потому что он рекурсивный. В Node это начинается примерно с 2000 уровней. Пример искусственный, но валидный. Chrome в таком случае просто перестаёт вкладывать элементы глубже 512 уровней, и так же можно сделать у себя:

```javascript
if (!closed && stack.length < 512) stack.push(element);
```

Со всеми правками выше (кроме сущностей, их я проверял отдельно) я ещё раз прогнал все 79 файлов: ничего не падает. Каждый пример из этого ответа теперь даёт то же дерево, что и браузер.

Ещё одно расхождение я не стал считать ошибкой: браузер всегда достраивает `html`, `head`, `body` и `tbody`, а твой парсер возвращает только то, что реально написано в разметке. Для такого задания это нормальный выбор, просто упомяни его в README рядом с пунктом про пробелы.

сделал автозакрытие li, p, td и других, вот buildTree. проверь, нет ли ошибок

```
function buildTree(tokens) {
    const root = { type: 'document', children:
[] };
    const stack = [root];

    for (const token of tokens) {
        if (token.type === 'startTag') {
            // закрываем то, что новый тег
закрывает неявно.
            // while, а не if: <tr> после <td>
закрывает и td, и предыдущий tr
            while
(AUTO_CLOSE[token.name]?.includes(stack[stack.l
ength - 1].tag)) {
                stack.pop();
            }
            if (
                stack[stack.length - 1].tag ===
'p' &&
                P_CLOSERS.includes(token.name)
            ) {
                stack.pop();
            }
        }

        // родителя берем только после
автозакрытия
        const parent = stack[stack.length - 1];

        if (token.type === 'startTag') {
            const element = {
                type: 'element',
                tag: token.name,
                attributes: token.attributes,
                children: []
            };
            parent.children.push(element);
            if
(!VOID_TAGS.includes(token.name))
stack.push(element);
        } else if (token.type === 'endTag') {
            const index =
stack.findLastIndex(element => element.tag ===
token.name);
            if (index > 0) stack.length =
```

```
index;
        } else if (token.type === 'text') {
            if (token.value.trim() === '')
continue;

            const last =
parent.children[parent.children.length - 1];
            if (last && last.type === 'text')
last.value += token.value;
            else parent.children.push({ type:
'text', value: token.value });
        } else {
            // комментарий или doctype кладем
в дерево как есть
            parent.children.push(token);
        }
    }

    return root;
}
```

Show more

Ran 2 commands

Простые случаи работают. На всех 79 файлах ничего не падает, и вот эти примеры дают то же дерево, что и браузер:

- `<li>a<li>b`
- `<td>a<td>b<tr>`
- `<dt>/<dd>`
- `<p>Intro<div>`
- вложенные списки

Но я нашёл одно падение и одну логическую ошибку.

1. Падение на теге `<constructor>`

```
<constructor>x</constructor>
TypeError: AUTO_CLOSE[token.name]?.includes is not a function
```

У любого объекта в JS есть унаследованное свойство `constructor`. Поэтому `AUTO_CLOSE['constructor']` возвращает функцию `Object`, а не `undefined`. `?.` защищает только от `null` и `undefined`, так что дальше вызывается `.includes`, которого у функции нет. Твой токенизер пропустит любое имя тега, которое начинается с

буквы, поэтому такой HTML дойдёт до buildTree. Исправление: проверять, что ключ действительно лежит в самом объекте:

```javascript
const rule = Object.hasOwn(AUTO_CLOSE, token.name) ? AUTO_CLOSE[token.name] : n
```

Второй вариант: создать объект через `Object.create(null)`, тогда у него нет прототипа.

2. Проверяется только верхний элемент стека

Если между старым `li` и новым стоит открытый `p` или `div`, `li` не закроется. А это валидный HTML:

```
<ul><li><p>One<li><p>Two</ul>
у тебя:   ul[li[p["One", li[p["Two"]]]]]
браузер:  ul[li[p["One"]], li[p["Two"]]]

<table><tr><td><p>A<td><p>B</table>
у тебя:   td[p["A", td[p["B"]]]]
браузер:  td[p["A"]], td[p["B"]]

<ul><li><div>a<li>b</ul>
у тебя:   li[div["a", li["b"]]]
браузер:  li[div["a"]], li["b"]
```

Браузер ищет закрываемый элемент вниз по стеку, а не только наверху. Но поиск останавливается на границе: для `li` это `ul`/`ol`, для `td` это `tr`/`table`. Без такой границы вложенный список закрыл бы внешний `li`. Это почти то же самое, только правило задаёт ещё и границу поиска:

```javascript
```

```
const AUTO_CLOSE = {
    li: { close: ['li'], stopAt: ['ul', 'ol', 'menu'] },
    dt: { close: ['dt', 'dd'], stopAt: ['dl'] },
    dd: { close: ['dt', 'dd'], stopAt: ['dl'] },
    option: { close: ['option'], stopAt: ['select', 'datalist', 'optgroup'] },
    optgroup: { close: ['option', 'optgroup'], stopAt: ['select'] },
    tr: { close: ['tr'], stopAt: ['table', 'thead', 'tbody', 'tfoot'] },
    td: { close: ['td', 'th'], stopAt: ['tr', 'table'] },
    th: { close: ['td', 'th'], stopAt: ['tr', 'table'] },
    thead: { close: ['thead', 'tbody', 'tfoot'], stopAt: ['table'] },
    tbody: { close: ['thead', 'tbody', 'tfoot'], stopAt: ['table'] },
    tfoot: { close: ['thead', 'tbody', 'tfoot'], stopAt: ['table'] }
};
// 'p' теперь тоже здесь, отдельная запись p: ['p'] больше не нужна
const P_CLOSERS = ['p', 'div', 'ul', 'ol', 'dl', 'table', 'h1', 'h2', 'h3', 'h4
    'section', 'article', 'header', 'footer', 'nav', 'main', 'aside', 'form', '
    'blockquote', 'hr', 'figure'];
const P_STOP_AT = ['button', 'table', 'td', 'th', 'caption', 'template', 'svg',

// идем вниз по стеку до границы и закрываем самый нижний подходящий элемент
// вместе со всем, что открыто над ним
function closeImplied(stack, close, stopAt) {
    let cut = -1;
    for (let j = stack.length - 1; j > 0; j--) {
        const tag = stack[j].tag;
        if (stopAt.includes(tag)) break;
        if (close.includes(tag)) cut = j;
    }
    if (cut !== -1) stack.length = cut;
}
```

А начало цикла в `buildTree` теперь такое:

javascript

```
    for (const token of tokens) {
        if (token.type === 'startTag') {
            if (P_CLOSERS.includes(token.name)) closeImplied(stack, ['p'], P_ST
            const rule = Object.hasOwn(AUTO_CLOSE, token.name) ? AUTO_CLOSE[tok
            if (rule) closeImplied(stack, rule.close, rule.stopAt);
        }

        // родителя берем только после автозакрытия
        const parent = stack[stack.length - 1];
        // ... дальше без изменений
```

Почему закрывается самый нижний подходящий элемент, а не первый найденный: при `<optgroup><option>1<optgroup>` новый `optgroup` должен закрыть и `option`, и предыдущий `optgroup`. Если остановиться на первом совпадении, закроется только `option`. Отдельный `while` при этом больше не нужен: `<tr>` после `<td>` находит `tr` под `td` и срезает стек сразу с этого места.

В правилах появилось то, чего, судя по коду, у тебя не было: `optgroup`, `thead`, `tbody`, `tfoot`. Без них `<thead><tr><th>H<tbody>` (тоже валидный HTML) кладёт `tbody` внутрь `th`.

С этой версией я прогнал те же примеры и сверил с parse5: все совпадают с браузером. Единственная разница в том, что браузер сам вставляет `tbody`, но это твоё осознанное решение. Все 79 файлов тоже проходят. По скорости проблем нет: цикл останавливается на первой границе. Даже при 3000 вложенных `div` (там `P_CLOSERS` проходит весь стек) это около 4.5 млн простых сравнений, доли секунды.

Что не связано с автозакрытием и осталось как было: `if (token.value.trim() === '') continue;` всё ещё выкидывает пробел в `<b>Hello</b> <i>world</i>` (пункт 3 из прошлого ответа).

Pasted text.txt

TXT

посмотри код, проверь его
и скажи мне есть ли там какие-то ошибки, и напомни дальнейшие этапы

Ran 2 commands ▸

Я прогнал код по всем 79 файлам и по отдельным примерам. Автозакрытие и `findTagEnd` в основном работают. Нашёл три ошибки в новом коде и одну проблему с

отладкой.

Ошибки

1. `findTagEnd` может съесть полдокумента. Кавычка открывается в любом месте тега, а не только после `=`. Если в значении без кавычек встретится апостроф, функция ищет закрывающий `'` по всему тексту дальше:

```
<a href=it's>link</a><p>don't</p><p>next</p>
сейчас:   a{ href: "it's", "link<": "", a: "", "<p": "", don: "", ... }
[p["next"]]
браузер:  a{ href: "it's" }["link"], p["don't"], p["next"]
```

Комментарий в конце функции обещает, что в таком случае берётся первый `>`, но это не так. Откат на `indexOf` срабатывает, только если второго `'` нет до самого конца текста. Браузер считает кавычку началом значения, только если она стоит сразу после `=` (пробелы между ними допустимы). Так и надо сделать:

javascript

```
function findTagEnd(text, start) {
    let quote = null; // какая кавычка сейчас открыта: " или ', null - никакая
    let prev = ''; // предыдущий непробельный символ вне кавычек

    for (let i = start; i < text.length; i++) {
        const char = text[i];

        if (quote) {
            if (char === quote) quote = null; // кавычка закрылась
        } else if ((char === '"' || char === "'") && prev === '=') {
            quote = char; // кавычка открывает значение, только если стоит сраз
        } else if (char === '>') {
            return i; // ">" вне кавычек - конец тега
        }

        if (!quote && !/\s/.test(char)) prev = char;
    }

    // кавычка не закрылась до конца текста - берем первый ">"
    return text.indexOf('>', start);
}
```

2. Закрывающий тег с атрибутами ничего не закрывает. Из `</div class="a">` получается имя `'div class="a"'`. `findLastIndex` его не находит, `div` остаётся открытым, и всё дальше попадает внутрь него:

```
<div class="a">x</div class="a"><p>after</p>
сейчас:   div["x", p["after"]]
браузер:  div["x"], p["after"]
```

Исправление такое же, как для открывающих тегов:

```javascript
const name = text
    .slice(i + 2, end)
    .match(/^[^\s\/>]*/)[0]
    .toLowerCase();
```

3. В `P_CLOSERS` не хватает тегов. `<p>Intro<details>...` и `<p>Intro<fieldset>` даже валидный HTML, но `details` или `fieldset` оказываются внутри `p`. Добавь в список:

```javascript
'address', 'details', 'dialog', 'fieldset', 'figcaption', 'hgroup',
'menu', 'search', 'summary', 'li', 'dd', 'dt'
```

`li`, `dd` и `dt` тоже закрывают открытый `p`: из `<p>a<li>b` браузер делает `p["a"], li["b"]`.

4. Отладочные `console.log` в конце файла (строки 326-332). Они срабатывают при каждой загрузке страницы, а ещё каждый раз при запуске `test_samples.js`. Перед сдачей их нужно удалить.

Со всеми четырьмя правками я прогнал всё ещё раз: примеры выше дают дерево как в браузере, из 79 файлов падает только `77_deep_nesting` (пункт 4 ниже).

Мелочи в комментариях: "омментарий" (строка 134), "текст до конца строки" вместо "до конца текста" (строка 184). Строки 61 и 62 говорят одно и то же.

Что осталось сделать

1. **Пробелы.** `trim() === ''` всё ещё выкидывает пробел в `<b>Hello</b> <i>world</i>`, и получается “Helloworld”. Нужно выбрасывать только пробельные узлы с переносом строки и не трогать содержимое `pre`.
2. **Сущности.** В твоём же `showExample1` получается `"© 2024"` вместо `"© 2024"`. Нужна функция `decodeEntities` для текста и значений атрибутов, но не для script и style.
3. SVG. `<svg><circle/><rect/></svg>` по-прежнему кладёт `rect` внутрь `circle`. Нужен флаг `selfClosing` в токене и проверка, что мы внутри `svg` или `math`.
4. **Глубина вложенности.** `if (!VOID_TAGS.includes(token.name) && stack.length < 512)`: после этого `test_samples.js` будет давать 79/79.
5. **Тесты с ожидаемым результатом.** Сейчас `test_samples.js` проверяет только "не падает". Стоит добавить 10-15 проверок "вход → ожидаемое дерево" на

встроенных `node:test` и `assert.deepStrictEqual`, без зависимостей. Кандидаты: автозакрытие li/p/td, script с `</div>` внутри, атрибуты всех видов, `>` в кавычках. Такие тесты в тестовом задании выглядят сильно, особенно раз тестов в твоих проектах пока почти нет.

6. **README с обоснованием структуры JSON.** Комментарии в задании прямо этого требуют: “be ready to provide reasoning behind such choice”. Что туда написать:
   - какие типы узлов есть (`document`, `element`, `text`, `comment`, `doctype`);
   - почему атрибуты хранятся объектом, а дети массивом (порядок детей важен, порядок атрибутов нет);
   - почему выбрасываются пробельные узлы;
   - почему не достраиваются `html`, `head`, `body`, `tbody`;
   - что сознательно не поддерживается (foster parenting, adoption agency, полная таблица сущностей).

   Это же пригодится, если на собеседовании попросят объяснить решения.
7. **Последняя проверка в браузере.** Открыть `index.html`, нажать оба примера и конвертацию, убедиться, что в консоли нет ошибок и лишних логов.

Пункты 1-4 займут по 5-10 минут каждый, весь код для них уже есть в предыдущих ответах. Больше всего времени и пользы дадут 5 и 6.

> Пробелы. trim() === '' всё ещё выкидывает пробел в Hello world, ...

переписал функцию, проверь

```
function buildTree(tokens) {
    const root = { type: 'document', children:
[] };
    const stack = [root];

    for (const token of tokens) {
        if (token.type === 'startTag') {
            if (P_CLOSERS.includes(token.name))
closeImplied(stack, ['p'], P_STOP_AT);
            // Object.hasOwn, а не
AUTO_CLOSE[name]: иначе тег <constructor>
найдет встроенное свойство объекта
            const rule =
Object.hasOwn(AUTO_CLOSE, token.name)
                ? AUTO_CLOSE[token.name]
                : null;
            if (rule) closeImplied(stack,
rule.close, rule.stopAt);
        }

        // родителя берем только после
автозакрытия
        const parent = stack[stack.length - 1];

        if (token.type === 'startTag') {
            const element = {
                type: 'element',
                tag: token.name,
                attributes: token.attributes,
                children: []
            };
            parent.children.push(element);
            if
(!VOID_TAGS.includes(token.name))
stack.push(element);
        } else if (token.type === 'endTag') {
            const index =
stack.findLastIndex(element => element.tag ===
token.name);
            if (index > 0) stack.length =
index;
        } else if (token.type === 'text') {
            // пробельный текст с переносом
строки - это отступы в коде, их выбрасываем.
            // одиночный пробел между словами
```

```
оставляем, внутри <pre> и <textarea> не
трогаем ничего
            const isIndent = token.value.trim()
=== '' && token.value.includes('\n');
            const keepsSpaces = stack.some(
                el => el.tag === 'pre' ||
el.tag === 'textarea'
            );
            if (isIndent && !keepsSpaces)
continue;
            const last =
parent.children[parent.children.length - 1];
            if (last && last.type === 'text')
last.value += token.value;
            else parent.children.push({ type:
'text', value: token.value });
        } else {
            // комментарий или doctype кладем
в дерево как есть
            parent.children.push(token);
        }
    }

    return root;
}
```

Show more

Ran 2 commands ▸

Логика верная. Я прогнал функцию по всем 79 файлам, и тот самый случай теперь работает: `<b>Hello</b> <i>world</i>` даёт `p[b["Hello"], " ", i["world"]]`. Отступы между блоками выбрасываются, содержимое `pre` и `textarea` не трогается. Падает по-прежнему только `77_deep_nesting`, это пункт 4 из прошлого списка.

Нашёл одну ошибку и одно ограничение, которое стоит описать в README.

Ошибка: `trim()` удаляет неразрывный пробел

В JS `trim()` убирает не только обычные пробелы, но и U+00A0 (неразрывный пробел, ` `). А в HTML пробельными считаются только пробел, таб, `\n`, `\r` и `\f`. Поэтому неразрывный пробел с переносом строки после него считается отступом и пропадает:

```
<td>[U+00A0]
</td>
сейчас:   td                  ячейка пустая
браузер:  td["\u00A0\n"]
```

Пока сущности не декодируются, это ловится только на "живом" символе U+00A0. Такой символ часто попадает в HTML из копипаста или визуальных редакторов. Но когда ты добавишь `decodeEntities`, так же начнёт пропадать и обычная табличная вёрстка вида `<td>\n   \n</td>`. Исправление: проверять регуляркой только пробельные символы HTML:

```javascript
const isIndent = /^[ \t\n\r\f]*$/.test(token.value) && token.value.includes('\n
```

Ограничение: перенос строки между инлайн-элементами

```html
<p>
  <a href="/">Home</a>
  <a href="/about">About</a>
</p>
```

Сейчас получается `p[a["Home"], a["About"]]`. Браузер покажет это как “Home About”: перенос между ссылками отрисуется как пробел, а в твоём JSON его уже нет.

Это не баг кода, а цена эвристики. Отличить "отступ, который ничего не значит" от "пробела между словами" можно только зная CSS, то есть блочный элемент или строчный. Правильного решения без CSS нет, поэтому просто опиши это в README. Примерно так: пробельные узлы с переносом строки считаются форматированием и выбрасываются, поэтому между инлайн-элементами на разных строках пробел теряется. Если такие пробелы важны, нужно убрать фильтр и хранить все узлы, как браузер.

Мелочи, которые браузер делает иначе (по желанию)

Первый перенос строки после `<pre>` и `<textarea>`. Браузер его отбрасывает: из `<pre>\n  code</pre>` получается `"  code"`, а у тебя `"\n  code"`. Это три строки в начале ветки `text`:

```javascript
let value = token.value;
// браузер отбрасывает один перенос строки сразу после <pre> и <textarea>
if ((parent.tag === 'pre' || parent.tag === 'textarea') &&
    parent.children.length === 0 && value.startsWith('\n')) {
    value = value.slice(1);
}
if (value === '') continue;
// дальше везде value вместо token.value
```

Окончания строк Windows (CRLF). Если HTML вставили из файла с CRLF, в JSON окажутся `"\r\n"`. Браузер перед разбором заменяет все `\r\n` и одиночные `\r` на `\n`.

Это одна строка в `html2json`, и после неё проверку на `\r` в регулярке можно не держать:

```javascript
function html2json(htmlText) {
    return buildTree(splitIntoTokens(htmlText.replace(/\r\n?/g, '\n')));
}
```

Со всеми правками выше я ещё раз проверил примеры: всё совпадает с браузером. `<pre>\r\nline1\r\nline2</pre>` даёт `pre["line1\nline2"]`, а `<pre>\n\nkept one</pre>` даёт `pre["\nkept one"]`: отбрасывается ровно один перенос.

Раз ты сейчас в этой функции, заодно можно закрыть пункт 4 (глубина вложенности): `if (!VOID_TAGS.includes(token.name) && stack.length < 512) stack.push(element);`. Тогда `test_samples.js` станет 79/79.

Pasted text.txt

TXT

посмотри мой код и скажи что осталось добавить, чтобы задание было выполнено на 100 процентов
распиши по этапам

Shared 2 files, ran 2 commands

Парсер в целом готов. Оба примера из задания (`showExample1` и `showExample2`) разбираются в правильное дерево, кроме `©`, и на 78 файлах из 79 ничего не падает. Остальное в основном вокруг кода. Текста задания я не видел, только код, поэтому этапы составлены по шаблону и по комментариям в нём. Если там есть другие требования, например репозиторий или деплой, пришли его.

Все правки ниже я применил к копии твоего кода и проверил: 12 тестов проходят, `test_samples.js` даёт 79/79.

Этап 1. Что заметит проверяющий (15 минут)

1.1. Кнопки примеров показывают не JSON. Сейчас `showExample1` и `showExample2` выводят в поле JSON текст “Comment 1 / Comment 2”. Скорее всего, это заглушка из шаблона, на месте которой должен быть твой результат. Проверяющий первым делом нажмёт эти кнопки. В обеих функциях замени `jsonContent` и вывод на:

```javascript
```

```
document.getElementById('html').value = htmlExample;
convertHtml2JsonAndSet();
```

Сами комментарии из шаблона перенеси в README, ответ на них будет там (этап 4).

1.2. Удали `console.log` в конце файла (строки 346-352). Они печатаются при каждой загрузке страницы.

1.3. Сущности. Это единственный дефект, видный прямо на первом примере: `"© 2024 My Website"` вместо `"© 2024 My Website"`. Добавь функцию:

javascript

```
const ENTITIES = {
    amp: '&', lt: '<', gt: '>', quot: '"', apos: "'", nbsp: '\u00A0',
    copy: '\u00A9', reg: '\u00AE', trade: '\u2122', hellip: '\u2026',
    mdash: '\u2014', ndash: '\u2013', laquo: '\u00AB', raquo: '\u00BB',
    euro: '\u20AC', middot: '\u00B7', times: '\u00D7'
};

function decodeEntities(str) {
    return str.replace(/&(#[xX][0-9a-fA-F]+|#\d+|[a-zA-Z]+);/g, (match, entity)
        if (entity[0] === '#') {
            const isHex = entity[1] === 'x' || entity[1] === 'X';
            const code = parseInt(entity.slice(isHex ? 2 : 1), isHex ? 16 : 10)
            // fromCodePoint бросает RangeError на числах больше 0x10FFFF
            const valid = code > 0 && code <= 0x10ffff && !(code >= 0xd800 && c
            return valid ? String.fromCodePoint(code) : '\uFFFD';
        }
        return Object.hasOwn(ENTITIES, entity) ? ENTITIES[entity] : match;
    });
}
```

И вызови её в трёх местах:

javascript

```
// parseAttributes
attributes[name] = decodeEntities(match[2] ?? match[3] ?? match[4] ?? '');

// ветка TEXT_TAGS: в textarea декодируем, в script и style нет
const raw = text.slice(i, textEnd);
tokens.push({ type: 'text', value: name === 'textarea' ? decodeEntities(raw) :

// последняя ветка else (обычный текст)
tokens.push({ type: 'text', value: decodeEntities(text.slice(i, end)) });
```

`Object.hasOwn` здесь нужен по той же причине, что и в `AUTO_CLOSE`: `&constructor;` иначе вернул бы функцию.

Этап 2. Оставшиеся случаи в парсере (10 минут)

2.1. SVG и глубина вложенности. Обе правки в одном месте:

javascript

```
// splitIntoTokens, вместо tokens.push({ type: 'startTag', name, attributes });
const selfClosing = /\/\s*$/.test(inside);
tokens.push({ type: 'startTag', name, attributes, selfClosing });

// рядом с остальными константами
const MAX_DEPTH = 512; // как в Chrome, иначе JSON.stringify падает на очень гл

// buildTree, вместо if (!VOID_TAGS.includes(token.name)) stack.push(element);
const inForeign = token.name === 'svg' || token.name === 'math' ||
    stack.some(el => el.tag === 'svg' || el.tag === 'math');
const closed = VOID_TAGS.includes(token.name) || (inForeign && token.selfClosin
if (!closed && stack.length < MAX_DEPTH) stack.push(element);
```

После этого `test_samples.js` даёт 79/79.

2.2. По желанию: первый перенос строки после `<pre>` и `<textarea>` (код был в прошлом ответе). Ещё есть опечатки в комментариях: "омментарий", "до конца строки", и одна и та же мысль повторена в строках 63 и 64.

Этап 3. Тесты с ожидаемым результатом (10 минут)

Положи в корень проекта оба файла:

- `html2json.test.js`: 12 тестов вида "вход → ожидаемое дерево". Покрыты атрибуты, `>` в кавычках, автозакрытие li/p/td, script, сущности, пробелы, комментарии, svg и оборванный тег.
- `load-html2json.js`: тот же приём с `vm`, вынесенный в функцию, чтобы не дублировать его в двух тестовых файлах.

Тесты на встроенном `node:test`, зависимостей нет. В `package.json` добавь:

```json
"scripts": {
  "test": "node --test",
  "test:samples": "node test_samples.js"
}
```

`npm test` сам находит файлы `*.test.js`. Тесты для сущностей и svg пройдут только после этапов 1 и 2. Остальные проходят уже на текущем коде.

Этап 4. README (30-40 минут, самое важное)

Второй комментарий в шаблоне прямо говорит, что решения по структуре JSON нужно обосновать. Разделы:

1. **Как запустить:** открыть `index.html`, `npm test`, `npm run test:samples`.
2. **Структура JSON:** пять типов узлов (`document`, `element`, `text`, `comment`, `doctype`) и короткий пример дерева.
3. **Почему так:**
   - `type` в каждом узле: обработчик дерева сразу знает, что перед ним.
   - Дети хранятся массивом, потому что их порядок важен.
   - Атрибуты хранятся объектом: порядок не важен, имена уникальны, доступ по `attributes.class`.
   - У void-элементов `children: []`, а не отсутствие поля, чтобы у всех элементов была одна форма.
4. **Как разбираются сложные случаи:** автозакрытие тегов, script/style/textarea, `>` в кавычках, оборванные теги, повторяющиеся атрибуты.
5. **Сознательные упрощения:**
   - не достраиваются `html`, `head`, `body`, `tbody`;
   - пробельные узлы с переносом строки выбрасываются, поэтому между инлайн-элементами на разных строках теряется пробел;
   - поддерживается только часть именованных сущностей;
   - не реализованы foster parenting и adoption agency;
   - ограничение глубины 512.
6. **Как тестировал:** 79 файлов в `html_samples` плюс тесты с ожидаемым деревом.

Пиши своими словами: README будут читать как твой текст, а на собеседовании по нему зададут вопросы. Раздел "почему так" лучше уметь рассказать вслух.

Этап 5. Финальная проверка (5 минут)

1. Открой `index.html`, нажми оба примера и конвертацию, вставь что-нибудь сломанное. В консоли не должно быть ни ошибок, ни логов.
2. `npm test` и `npm run test:samples` зелёные.
3. Проверь, что в репозитории нет лишнего: `gen_samples.py`, если он попал в папку, можно оставить только при условии, что README объясняет, откуда взялись

примеры.

4. По желанию: GitHub Pages, чтобы проверяющий открыл страницу по ссылке без клонирования.

Html2json.test
JS

Load html2json
JS

Download all

все правки внес, этап 1 и 2

у меня уже есть файл с тестами test_samples.js
проанализируй его и скажи что можно улучшить

```
import fs from 'node:fs';
import path from 'node:path';
import vm from 'node:vm';
import { fileURLToPath } from 'node:url';

const __dirname =
path.dirname(fileURLToPath(import.meta.url));

// html2json.js - обычный скрипт без export,
поэтому выполняем его через vm
// и забираем функцию последним выражением
const libPath = path.join(__dirname,
'html2json.js');
const code = fs.readFileSync(libPath,
'utf8');
const html2json =
vm.runInThisContext(`${code}\n;html2json;`, {
    filename: libPath
});

if (typeof html2json !== 'function') {
    console.error('html2json не найдена в
html2json.js');
    process.exit(1);
}

const samplesDir = path.join(__dirname,
'html_samples');
const files = fs
    .readdirSync(samplesDir)
    .filter(f =>
f.toLowerCase().endsWith('.html'))
    .sort();

if (files.length === 0) {
    console.log('В html_samples нет .html
файлов');
    process.exit(0);
}

let failed = 0;

for (const file of files) {
    const html =
fs.readFileSync(path.join(samplesDir, file),
```

```
'utf8');
    try {
        const result = html2json(html);
        // проверяем, что результат реально
сериализуется (нет циклических ссылок и т.п.)
        JSON.stringify(result);
        console.log(`OK    ${file}`);
    } catch (err) {
        failed++;
        console.error(`FAIL  ${file}\n
${err.stack}`);
    }
}

console.log(`\n${files.length -
failed}/${files.length} passed`);
process.exitCode = failed ? 1 : 0;
```

Show more

Shared a file, ran 5 commands ▸

Я переписал `test_samples.js`. Он проверяет на всех 79 файлах, что парсер не падает, не зависает, отдаёт дерево правильной формы и не меняет результат незаметно для тебя. Уже на первом прогоне новый скрипт нашёл две ошибки в парсере, которые старый пропускал.

Что было не так в старой версии

1. **Зависание повесило бы и сам тест.** Главная ошибка, которую мы ловили (файл 21), была бесконечным циклом. На ней старый скрипт просто висит навсегда без сообщения. `try/catch` не прерывает цикл, и таймаут в `node:test` тоже: синхронный код он не останавливает.
2. `html2json`**, который ничего не возвращает, проходит тест.** `JSON.stringify(undefined)` не бросает ошибку, а возвращает `undefined`. Если в функции потеряется `return`, все 79 файлов будут OK.
3. **Не проверяется, что дерево правильное.** Проверка только на "не упало" не видит текст в `<br>`, тег в верхнем регистре или узел без `children`.
4. **Не видно, что дерево изменилось.** Ты правишь автозакрытие, а оно ломает разбор таблиц. Старый скрипт этого не заметит.
5. **Пустая папка считается успехом.** `process.exit(0)` при отсутствии файлов скрывает ошибку в пути.

Что изменилось

Таймаут. Парсер выполняется в отдельном vm-контексте и вызывается с `{ timeout: 2000 }`. Таймаут vm прерывает даже `while (true) {}`. Я проверил это на

подложенной зависающей функции: `FAIL 21_unterminated_tag_eof.html, завис: не закончил за 2000 мс`, после чего скрипт идёт дальше.

Проверка формы дерева работает на любом HTML без ожидаемых результатов:

- корень `document`;
- у элемента непустой `tag` в нижнем регистре, `attributes` объект со строковыми значениями, `children` массив;
- у void-элементов нет детей;
- нет пустых текстовых узлов и двух текстовых подряд;
- `doctype` только на верхнем уровне.

Список void-тегов берётся из самого парсера через тот же vm-контекст, так что дублировать его не нужно.

Снапшоты. При первом запуске результат каждого файла записывается в `html_samples/__snapshots__/NN.json`. При следующих запусках новый результат сравнивается с записанным. Если дерево изменилось, скрипт показывает путь до места расхождения. Пример, когда я нарочно убрал `'p'` из `P_CLOSERS`:

```
FAIL  51_implied_end_p.html
      дерево отличается от снапшота: document > [2] <p> > [1] <p>: было
(нет), стало <p>
```

Если изменение сделано специально, запусти `node test_samples.js --update`, и снапшоты перезапишутся. Снапшоты закоммить в репозиторий: это и есть ожидаемые результаты для 79 случаев. После первой записи один раз просмотри их глазами, потому что они фиксируют текущее поведение вместе с возможными ошибками.

Время. Если файл разбирается дольше 200 мс, рядом с OK появляется `МЕДЛЕННО`. Это ловит квадратичную сложность на `78_large_file`.

Две ошибки, которые нашла проверка формы

1. Повторный doctype попадает внутрь body (`07_doctype_duplicate`). Браузер такой doctype игнорирует. В `buildTree` вынеси doctype из общей ветки:

javascript

```
        } else if (token.type === 'doctype') {
            // doctype имеет смысл только в начале документа, повторный браузер
            const hasElements = root.children.some(node => node.type === 'eleme
            if (stack.length === 1 && !hasElements) root.children.push(token);
        } else {
            // комментарий
            parent.children.push(token);
        }
```

2. BOM становится текстовым узлом (`09_bom`). Первым узлом документа оказывается невидимый текст `"\uFEFF"`. Браузер отбрасывает BOM ещё до разбора, поэтому сделай так же в `html2json`:

```javascript
function html2json(htmlText) {
    // BOM в начале файла браузер отбрасывает еще до разбора, как и \r\n -> \n
    const normalized = htmlText.replace(/^\uFEFF/, '').replace(/\r\n?/g, '\n');
    return buildTree(splitIntoTokens(normalized));
}
```

С обеими правками: 79/79, и 12 тестов из `html2json.test.js` тоже проходят.

Порядок запуска

1. Внеси две правки в парсер.
2. Замени `test_samples.js` на новый и запусти: снапшоты запишутся.
3. Просмотри несколько `.json` в `__snapshots__`, особенно 51-63 (автозакрытие и таблицы).
4. После этого любой следующий запуск сравнивает с ними.

Для README пригодится разница между двумя тестовыми файлами. `html2json.test.js` описывает, как должно быть: ожидаемые деревья там написаны руками. `test_samples.js` фиксирует, как есть сейчас, на 79 файлах и ловит падения, зависания и случайные изменения.

Test samples
JS

test_samples.js получился сложнее самого парсера
сделай проще, до 60-70 строк:
прогнать все .html из html_samples и показать OK или FAIL по каждому
FAIL если упал, завис (таймаут оставь) или вернул не document
проверить что JSON.stringify(result, null, 2) не падает

снапшоты и validateTree убери
и объясни каждую часть простыми словами

Write simplified test_samples.js and check timeout, wrong result and cycle c: Stopped

Claude's response was interrupted. Edit prompt Try again

test_samples.js получился сложнее самого парсера
сделай проще, до 60-70 строк:
прогнать все .html из html_samples и показать OK или FAIL по каждому
FAIL если упал, завис (таймаут оставь) или вернул не document
проверить что JSON.stringify(result, null, 2) не падает
снапшоты и validateTree убери
и объясни каждую часть простыми словами

файла html2json.test.js у меня нет и создавать его не нужно

Adjusting the file setup after the rejected tool call.
Вот упрощённая версия, 61 строка:

javascript

```
import fs from 'node:fs';
import path from 'node:path';
import vm from 'node:vm';
import { fileURLToPath } from 'node:url';

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const TIMEOUT_MS = 2000; // если разбор дольше - считаем, что парсер завис

// 1. Загружаем парсер. В html2json.js нет export, поэтому выполняем файл
//    в отдельном vm-контексте и потом вызываем html2json внутри него
const context = vm.createContext({});
const libPath = path.join(__dirname, 'html2json.js');
vm.runInContext(fs.readFileSync(libPath, 'utf8'), context, { filename: libPath

// 2. Вызов с таймаутом: vm прервет даже бесконечный цикл
function parse(html) {
    context.input = html;
    return vm.runInContext('html2json(input)', context, { timeout: TIMEOUT_MS }
}

// 3. Список примеров
const samplesDir = path.join(__dirname, 'html_samples');
const files = fs
    .readdirSync(samplesDir)
    .filter(f => f.endsWith('.html'))
    .sort();

if (files.length === 0) {
    console.error('В html_samples нет .html файлов');
    process.exit(1);
}

// 4. Прогоняем каждый файл
let failed = 0;

for (const file of files) {
    const html = fs.readFileSync(path.join(samplesDir, file), 'utf8');

    try {
        const result = parse(html);

        if (result?.type !== 'document') {
            throw new Error(`вернул не document, а ${JSON.stringify(result)?.sl
        }
```

```
        JSON.stringify(result, null, 2); // упадет на циклических ссылках или о
        console.log(`OK    ${file}`);
    } catch (err) {
        failed++;
        const reason =
            err?.code === 'ERR_SCRIPT_EXECUTION_TIMEOUT'
                ? `завис: не закончил за ${TIMEOUT_MS} мс`
                : (err?.message ?? err);
        console.error(`FAIL  ${file}\n      ${reason}`);
    }
}

// 5. Итог. Код выхода 1, если что-то упало
console.log(`\n${files.length - failed}/${files.length} passed`);
process.exitCode = failed ? 1 : 0;
```

Что делает каждая часть

Импорты и `__dirname`. `fs` читает файлы, `path` склеивает пути, `vm` запускает чужой код. `__dirname` это папка, где лежит сам тест. В режиме `type: module` такой переменной нет, поэтому её собирают из `import.meta.url`. Благодаря ей тест находит `html2json.js` и `html_samples`, из какой бы папки его ни запустили.

1. Загрузка парсера. `import` для `html2json.js` не подходит: в нём нет `export`, а добавлять его нельзя, иначе сломается подключение в `index.html`. Поэтому файл читается как обычный текст и выполняется внутри `vm.createContext`. Это отдельная "песочница" со своими глобальными переменными, как маленькое окно браузера. После выполнения в ней появляется функция `html2json`.

2. Функция `parse` **с таймаутом.** HTML кладётся в песочницу переменной `input`, и там же выполняется строка `html2json(input)`. Ради опции `timeout` и нужна песочница. Если парсер ушёл в бесконечный цикл, через 2 секунды vm его принудительно останавливает и бросает ошибку с кодом `ERR_SCRIPT_EXECUTION_TIMEOUT`. Обычный `try/catch` так не умеет: зависший цикл до него просто не дойдёт.

3. Список файлов. Берутся все `.html` из `html_samples` и сортируются, чтобы порядок был 01, 02, 03 и так далее. Если файлов нет, это ошибка: скорее всего, неправильный путь. Тест не должен в таком случае писать "всё прошло".

4. Цикл по файлам. Для каждого файла три проверки, любая из них ведёт в `catch` и даёт FAIL:

- **Упал.** Парсер бросил ошибку, например `RangeError` из `String.fromCodePoint`.
- **Вернул не** document. Если в функции потеряется `return`, придёт `undefined`. Это нужно ловить отдельно: `JSON.stringify(undefined)` ошибку не бросает. `?.` нужен, чтобы проверка сама не упала на `undefined`.
- **Не сериализуется.** `JSON.stringify(result, null, 2)` делает то же, что и твоя кнопка конвертации. Если дерево содержит ссылку само на себя или оно слишком

глубокое, ошибка будет здесь, а не у проверяющего на странице.

В `catch` выбирается понятный текст причины: для таймаута "завис", для остального сообщение ошибки. Запись `err?.message ?? err` нужна на случай, если кто-то бросил не `Error`, а просто строку.

5. Итог. Печатается количество прошедших файлов, например `79/79 passed`. `process.exitCode = 1` при ошибках означает, что команда завершилась неуспешно. Это видят `npm` и GitHub Actions, если когда-нибудь подключишь CI.

Раз `html2json.test.js` нет, в `package.json` хватит одного скрипта: `"test": "node test_samples.js"`.

Правки для повторного doctype и BOM из прошлого ответа в парсере всё равно стоит оставить, они от проверки формы дерева не зависят.

Pasted text.txt

TXT

нужно написать два README, на английском, коротко и по делу

1. корневой README.md вместо описания задания. первой строкой ссылка на оригинал задания: https://github.com/jito-dev/jito-intern-test-task
   дальше: как запустить (index.html в браузере, npm test без npm install), формат JSON с примером, почему такой формат, как устроен парсер (splitIntoTokens и buildTree), что обрабатывается, какие осознанные решения и ограничения, что проверяют тесты, как я использовал AI (чаты лежат в ai_help/)
2. в html_samples/README.md добавь в начало пояснение, что там описано поведение браузера, а не ожидаемый вывод html2json, и что отличия описаны в корневом README

пиши только то, что реально делает код, ничего не придумывай. вот код:

test_samples.js

```
import fs from 'node:fs';
import path from 'node:path';
import vm from 'node:vm';
import { fileURLToPath } from 'node:url';

const __dirname =
path.dirname(fileURLToPath(import.meta.url));
const TIMEOUT_MS = 2000; // если разбор дольше - считаем, что
парсер завис

// 1. Загружаем парсер. В html2json.js нет export, поэтому
выполняем файл
//    в отдельном vm-контексте и потом вызываем html2json
внутри него
const context = vm.createContext({});
const libPath = path.join(__dirname, 'html2json.js');
vm.runInContext(fs.readFileSync(libPath, 'utf8'), context, {
    filename: libPath
});

// 2. Вызов с таймаутом: vm прервет даже бесконечный цикл
function parse(html) {
    context.input = html;
    return vm.runInContext('html2json(input)', context, {
timeout: TIMEOUT_MS });
}

// 3. Список примеров
const samplesDir = path.join(__dirname, 'html_samples');
const files = fs
    .readdirSync(samplesDir)
    .filter(f => f.endsWith('.html'))
    .sort();

if (files.length === 0) {
    console.error('В html_samples нет .html файлов');
    process.exit(1);
}

// 4. Прогоняем каждый файл
let failed = 0;

for (const file of files) {
    const html = fs.readFileSync(path.join(samplesDir, file),
'utf8');
```

```
    try {
        const result = parse(html);

        if (result?.type !== 'document') {
            throw new Error(
                `вернул не document, а
${JSON.stringify(result)?.slice(0, 80)}`
            );
        }

        JSON.stringify(result, null, 2); // упадет на
циклических ссылках или очень глубоком дереве
        console.log(`OK    ${file}`);
    } catch (err) {
        failed++;
        const reason =
            err?.code === 'ERR_SCRIPT_EXECUTION_TIMEOUT'
                ? `завис: не закончил за ${TIMEOUT_MS} мс`
                : (err?.message ?? err);
        console.error(`FAIL  ${file}\n      ${reason}`);
    }
}

// 5. Итог. Код выхода 1, если что-то упало
console.log(`\n${files.length - failed}/${files.length}
passed`);
process.exitCode = failed ? 1 : 0;
```

Show more

Created a file, shared 2 files, ran 3 commands ▸
Оба README готовы, на английском.

Корневой `README.md` идёт в нужном порядке: ссылка на задание, запуск, формат JSON с примером, почему такой формат, устройство `splitIntoTokens` и `buildTree`, что обрабатывается, решения и ограничения, тесты, использование AI. Пример JSON в нём я не писал вручную, а получил, прогнав твой текущий `html2json.js`. Каждое утверждение я сверил с кодом. Например, в ограничениях есть то, что код действительно делает иначе, чем браузер:

- `viewBox` в SVG становится `viewbox`;
- `&copy` без `;` не декодируется;
- `<?xml ...?>` попадает в текст;
- `title` и `iframe` разбираются как обычный HTML;
- при парсинге с ошибкой возвращается `{ type: "error" }`.

`html_samples/README.md`: в начало добавлено пояснение на английском. Там сказано, что комментарии и таблица описывают поведение браузера, а не ожидаемый вывод `html2json`, что отличия перечислены в корневом README (со ссылкой на раздел) и что дальше текст на русском. Остальная часть файла не тронута.

Перед коммитом проверь четыре вещи:

1. `showExample1` и `showExample2` **всё ещё выводят заглушку** “Comment 1 / Comment 2”, а не JSON. Правку 1.1 из этапа 1 в этом коде я не вижу. Поэтому в README про кнопки примеров ничего не написано. Если поменяешь их на `convertHtml2JsonAndSet()`, можно добавить строку в раздел “How to run”.
2. `npm test` работает, только если в `package.json` есть `"test": "node test_samples.js"`. Сам `package.json` я не видел.
3. **Раздел** “How I used AI” я составил по этому чату: примеры, объяснение регулярки, ревью, найденные баги, тестовый скрипт. Если в `ai_help/` есть чаты с другими инструментами или ты делал с AI что-то ещё, поправь список. Этот раздел будут читать как твои слова.
4. **Число файлов** в `html_samples/README.md` указано как 79. Если ты что-то удалял или добавлял, исправь.

Readme
Document · MD

Readme
Document · MD

Download all

Pasted text.txt
TXT

переведи тут все на английский

Created a file, shared a file, ran a command
Перевёл весь `html_samples/README.md` на английский: вступление, оба абзаца и таблицу (все 79 строк). Строку “The descriptions below are in Russian” я убрал, она больше не нужна. Содержание не менял, только язык.

Но комментарии в начале самих `.html` файлов всё ещё на русском, а пояснение в README ссылается на них (“the comment at the top of each file”). Могу перевести и их. Я сгенерирую файлы заново тем же скриптом с английскими описаниями, а HTML внутри не изменится. Скажи, если нужно.

Readme

Document · MD